

PyAthena

AWS Wrangler

```
In [2]: import boto3
import awswrangler as wr

def athena_auth():
    boto3.setup_default_session(
        aws_access_key_id=aws_access_key_id,
        aws_secret_access_key=aws_secret_access_key,
        region_name=region_name
    )

    return {
        "database": "s3_atmos_stclair",
        "workgroup": "primary",
        "s3_output": "s3://halight-athena-query-results/stclair/"
    }

ATHENA_CFG = athena_auth()
```

```
In [3]: tables = wr.catalog.tables(database="s3_atmos_stclair")
print(tables)
```

	Database	Table Description \
0	s3_atmos_stclair	achievement_user_completions
1	s3_atmos_stclair	achievements
2	s3_atmos_stclair	banner_clicks
3	s3_atmos_stclair	bdg_user_completions
4	s3_atmos_stclair	certification_users
5	s3_atmos_stclair	comment_likes
6	s3_atmos_stclair	community_comments
7	s3_atmos_stclair	country_states
8	s3_atmos_stclair	instances
9	s3_atmos_stclair	instances_v
10	s3_atmos_stclair	li_attempts
11	s3_atmos_stclair	li_stats_agg_all
12	s3_atmos_stclair	li_stats_agg_all_v
13	s3_atmos_stclair	mi_user_mission_step_completions
14	s3_atmos_stclair	offboarded_instances
15	s3_atmos_stclair	poll_user_completions
16	s3_atmos_stclair	rank_user_completions
17	s3_atmos_stclair	rcs_historic_users
18	s3_atmos_stclair	rcs_historic_users_v
19	s3_atmos_stclair	rcs_lifetime_stats_by_month_v
20	s3_atmos_stclair	rcs_user_lifetimes_no_stats_v
21	s3_atmos_stclair	reward_prizes
22	s3_atmos_stclair	reward_user_claims
23	s3_atmos_stclair	reward_user_claims_v
24	s3_atmos_stclair	rewards
25	s3_atmos_stclair	sale_user_submissions
26	s3_atmos_stclair	snippet_view_logs
27	s3_atmos_stclair	snippets
28	s3_atmos_stclair	survey_user_completions
29	s3_atmos_stclair	test2_v
30	s3_atmos_stclair	test_v
31	s3_atmos_stclair	users

	TableType	Columns \
0	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
1	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
2	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
3	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
4	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
5	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
6	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
7	EXTERNAL_TABLE	country_short_code, country_name, state_name, ...
8	EXTERNAL_TABLE	instance_id, client_name, site_name, site_url,...
9	VIRTUAL_VIEW	instance_id, client_name, region_name, site_na...
10	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
11	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
12	VIRTUAL_VIEW	client_name, region_name, site_name, product_n...
13	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
14	EXTERNAL_TABLE	instance_id, client_name, site_name, site_url,...
15	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
16	EXTERNAL_TABLE	instance_id, generated_for_date, generated_ts,...
17	EXTERNAL_TABLE	instance_id, aggregated_for_date, user_id, pre...
18	VIRTUAL_VIEW	instance_id, client_name, region_name, site_na...
19	VIRTUAL_VIEW	instance_id, client_name, site_name, region_na...
20	VIRTUAL_VIEW	instance_id, user_id, start_date, end_date, st...

```

21 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
22 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
23 VIRTUAL_VIEW instance_id, client_name, region_name, site_na...
24 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
25 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
26 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
27 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
28 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...
29 VIRTUAL_VIEW test
30 VIRTUAL_VIEW test
31 EXTERNAL_TABLE instance_id, generated_for_date, generated_ts,...

```

Partitions

```

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31

```

Problem Statement

"We are predicting whether a user will make a claim within 7 days of using a product/training portal."

This is a classification problem (Yes/No).

Later, we could even turn it into regression if we want to predict the exact number of days until a claim

```
In [4]: df = wr.athena.read_sql_query(
        sql="SELECT * FROM reward_user_claims_v",
        **ATHENA_CFG,
        ctas_approach=False
    )
```

```
In [5]: df.head()
```

Out[5]:

	instance_id	client_name	region_name	site_name	product_name
0	EA53680F4661F0DE3467171D00E5B6C8	Electrolux	<NA>	Lära	Atmosphere
1	EA53680F4661F0DE3467171D00E5B6C8	Electrolux	<NA>	Lära	Atmosphere
2	A17B5D1C84AC7093025B72EB2653DA56	Lenovo	<NA>	Lenovo Training Portal	Atmosphere
3	A17B5D1C84AC7093025B72EB2653DA56	Lenovo	<NA>	Lenovo Training Portal	Atmosphere
4	A17B5D1C84AC7093025B72EB2653DA56	Lenovo	<NA>	Lenovo Training Portal	Atmosphere

5 rows × 59 columns



```
In [ ]: 0
sns.set_style("whitegrid")
pd.set_option("display.max_columns", None)
```

```
In [8]: df.shape
df.info()
df.describe(include="all")
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3585200 entries, 0 to 3585199
Data columns (total 59 columns):
 #   Column                                Dtype
---  -
 0   instance_id                          string
 1   client_name                          string
 2   region_name                          string
 3   site_name                            string
 4   product_name                         string
 5   instance_launch_date                 object
 6   generated_ts                         datetime64[ns]
 7   claim_id                             string
 8   claimed_ts                           datetime64[ns]
 9   last_updated_ts                      datetime64[ns]
10  achievement_id                        string
11  points_earned                         Int64
12  reward_id                             string
13  reward_title                          string
14  reward_created_ts                     datetime64[ns]
15  reward_type                           string
16  reward_is_active                      Int32
17  reward_quantity_total                 Int64
18  prize_id                              string
19  prize_title                           string
20  prize_monetary_value                  float32
21  prize_created_ts                      datetime64[ns]
22  prize_point_value                     Int64
23  prize_quantity                        Int32
24  prize_requires_user_info              Int32
25  prize_part_number                     string
26  user_id                               string
27  user_account_created_ts               datetime64[ns]
28  user_account_confirmed_ts             datetime64[ns]
29  user_account_merged_ts                datetime64[ns]
30  user_account_deleted_ts               datetime64[ns]
31  user_account_status                   string
32  user_verification_status               string
33  is_self_registration                  Int32
34  referred_by_user_id                   string
35  user_first_login_ts                   datetime64[ns]
36  user_last_login_ts                     datetime64[ns]
37  user_last_online_ts                   datetime64[ns]
38  user_type                             string
39  user_hierarchy_title                   string
40  user_retailer                         string
41  user_country_state                     string
42  user_location_country_code             string
43  user_location_state                     string
44  user_location_id                       string
45  user_location_title                     string
46  user_location_code                     string
47  user_location_longitude                 float32
48  user_location_latitude                 float32
49  user_location_city                     string
50  user_location_zipcode                  string

```

```
51 user_location_address_line1 string
52 user_location_address_line2 string
53 user_location_metadata_json string
54 user_country_code           string
55 user_country_name           string
56 user_rank_number            Int32
57 user_rank_title             string
58 user_last_li_completion_ts   datetime64[ns]
dtypes: Int32(5), Int64(3), datetime64[ns](13), float32(3), object(1), string(34)
memory usage: 1.5+ GB
```

Out[8]:

	instance_id	client_name	region_name	site_name	product
--	-------------	-------------	-------------	-----------	---------

count	3585200	3585200	2888256	3585200	3
unique	35	29	5	30	3
top	7C673923D3074C1589239BD124276E45	Motorola	NA	motorola insiders	Atm
freq	2609023	2778395	2713451	2778395	3
mean	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN

Datetime parsing

```
In [9]: date_cols = ["instance_launch_date", "generated_ts", "claimed_ts", "last_updated_ts"]
for col in date_cols:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")
```

Missing values

```
In [10]: missing = df.isna().mean().sort_values(ascending=False)
missing.head(20)
```

```
Out[10]: user_account_merged_ts      0.998844
user_location_address_line2      0.967172
prize_part_number                0.948247
user_location_longitude          0.914613
user_location_latitude           0.914613
last_updated_ts                 0.775384
user_account_deleted_ts          0.747008
prize_monetary_value             0.720285
prize_point_value               0.718491
prize_created_ts                0.718491
prize_quantity                  0.718491
prize_requires_user_info         0.718491
prize_id                        0.718491
prize_title                      0.718491
points_earned                   0.635456
user_rank_number                 0.629528
user_rank_title                  0.629528
user_location_metadata_json      0.401096
user_location_code               0.383371
referred_by_user_id              0.337353
dtype: float64
```

```
In [11]: drop_cols = missing[missing > 0.8].index
df.drop(columns=drop_cols, inplace=True)
```

```
In [14]: # Sort by instance and claimed timestamp
df = df.sort_values(["instance_id", "claimed_ts"])

# Next claim timestamp per instance
df["next_claim_ts"] = df.groupby("instance_id")["claimed_ts"].shift(-1)

# Days to next claim
df["days_to_next_claim"] = (df["next_claim_ts"] - df["claimed_ts"]).dt.days

# Target: claim within 7 days (1 = yes, 0 = no)
df["will_claim_next_7_days"] = (df["days_to_next_claim"].le(7).fillna(0).astype(int))

# Drop rows without claimed_ts
df = df.dropna(subset=["claimed_ts"])
```

```
In [15]: # Drop rows with missing claimed_ts or instance_launch_date

df = df.dropna(subset=["claimed_ts", "instance_launch_date"])
```

Feature engineering

```
In [16]: df["days_to_claim"] = (df["claimed_ts"] - df["instance_launch_date"]).dt.days

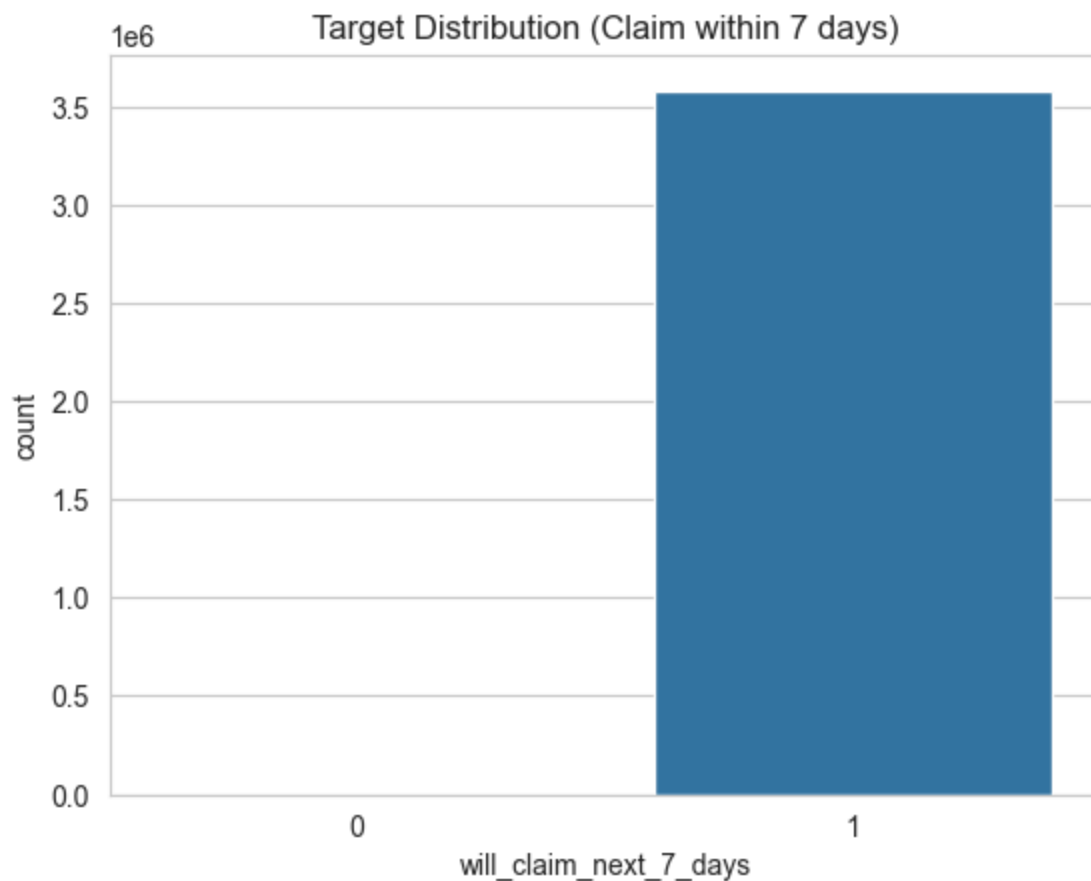
df["claim_hour"] = df["claimed_ts"].dt.hour

df["claim_day_of_week"] = df["claimed_ts"].dt.dayofweek

df["days_since_last_claim"] = (df["claimed_ts"] - df.groupby("instance_id")["claime
```

```
df["total_claims_so_far"] = df.groupby("instance_id").cumcount()
```

```
In [17]: sns.countplot(x="will_claim_next_7_days", data=df)
plt.title("Target Distribution (Claim within 7 days)")
plt.show()
```

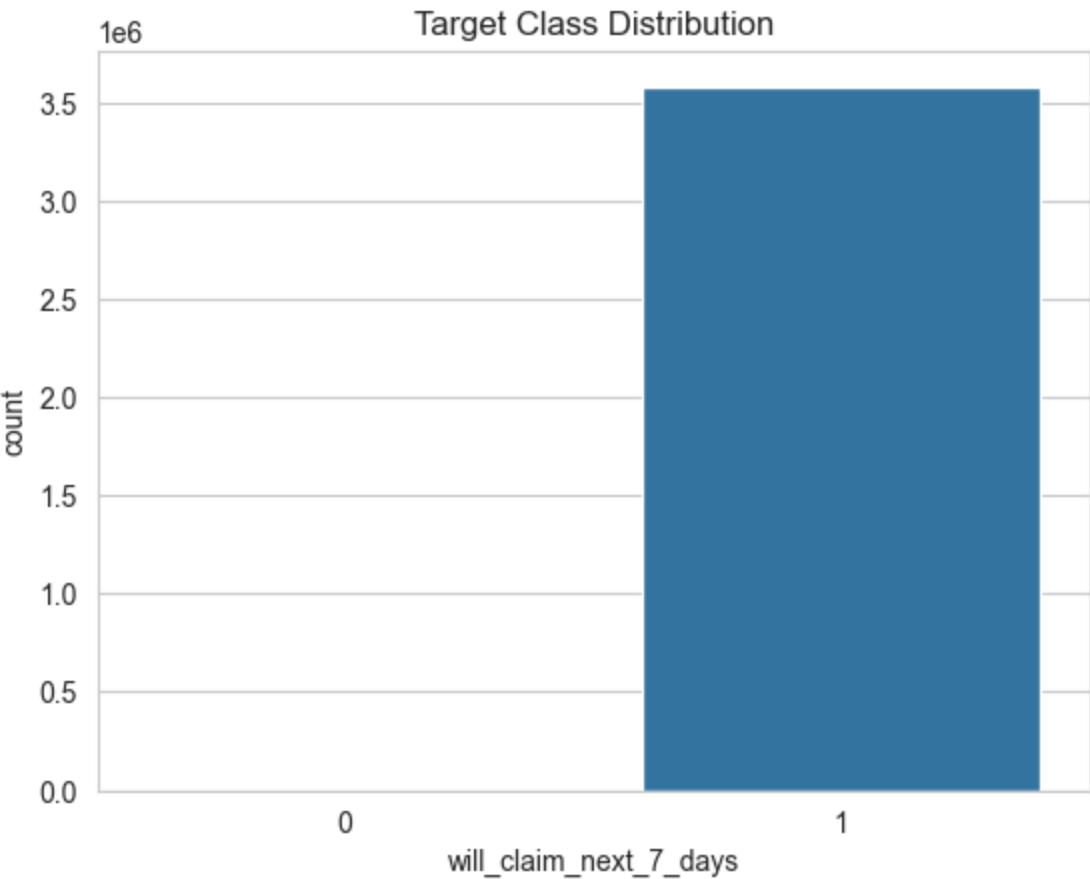


```
In [18]: #Check class balance

print(df["will_claim_next_7_days"].value_counts(normalize=True))

sns.countplot(x="will_claim_next_7_days", data=df)
plt.title("Target Class Distribution")
plt.show()
```

```
will_claim_next_7_days
1    0.999906
0    0.000094
Name: proportion, dtype: float64
```

Numerical EDA

```
In [20]: num_cols = [
          "days_to_claim",
          "days_since_last_claim",
          "total_claims_so_far",
          "claim_hour"
        ]

df[num_cols].describe()
```

Out[20]:

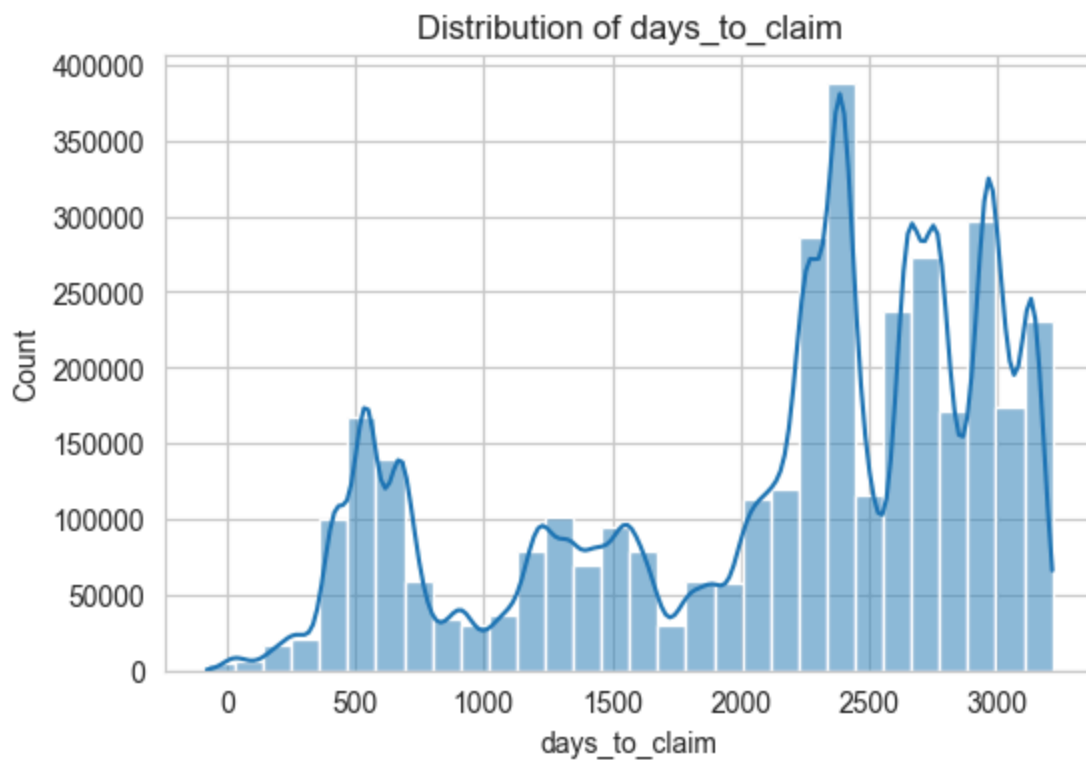
	days_to_claim	days_since_last_claim	total_claims_so_far	claim_hour
count	3.585200e+06	3.585165e+06	3.585200e+06	3.585200e+06
mean	2.114098e+03	4.408165e-03	9.841785e+05	1.337432e+01
std	8.444681e+02	7.610595e-01	8.311767e+05	4.879888e+00
min	-7.700000e+01	0.000000e+00	0.000000e+00	0.000000e+00
25%	1.506000e+03	0.000000e+00	1.752088e+05	1.100000e+01
50%	2.374000e+03	0.000000e+00	8.164225e+05	1.300000e+01
75%	2.770000e+03	0.000000e+00	1.712722e+06	1.600000e+01
max	3.213000e+03	8.320000e+02	2.609022e+06	2.300000e+01

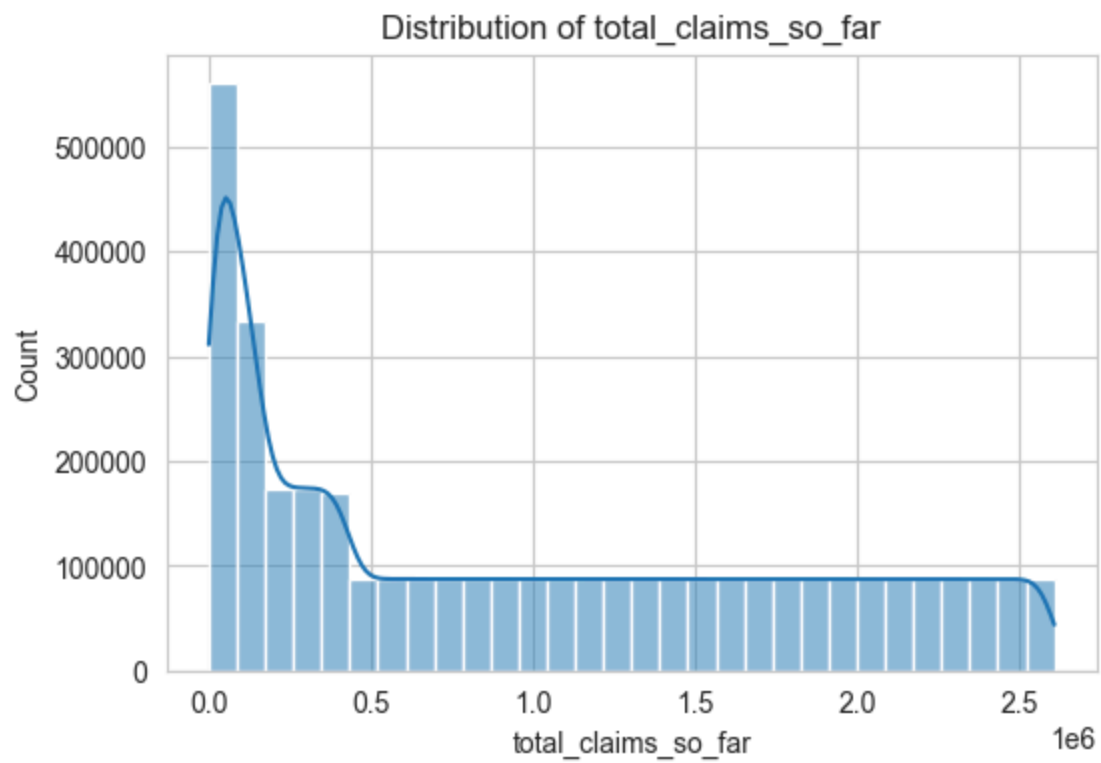
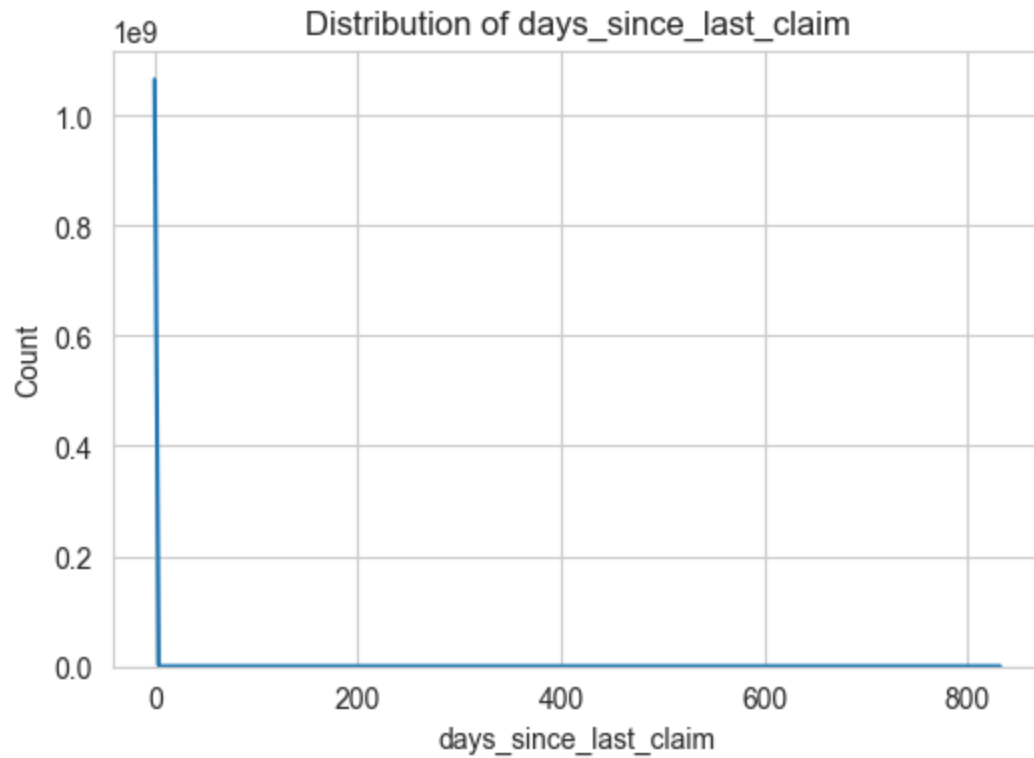
Univariate Analysis

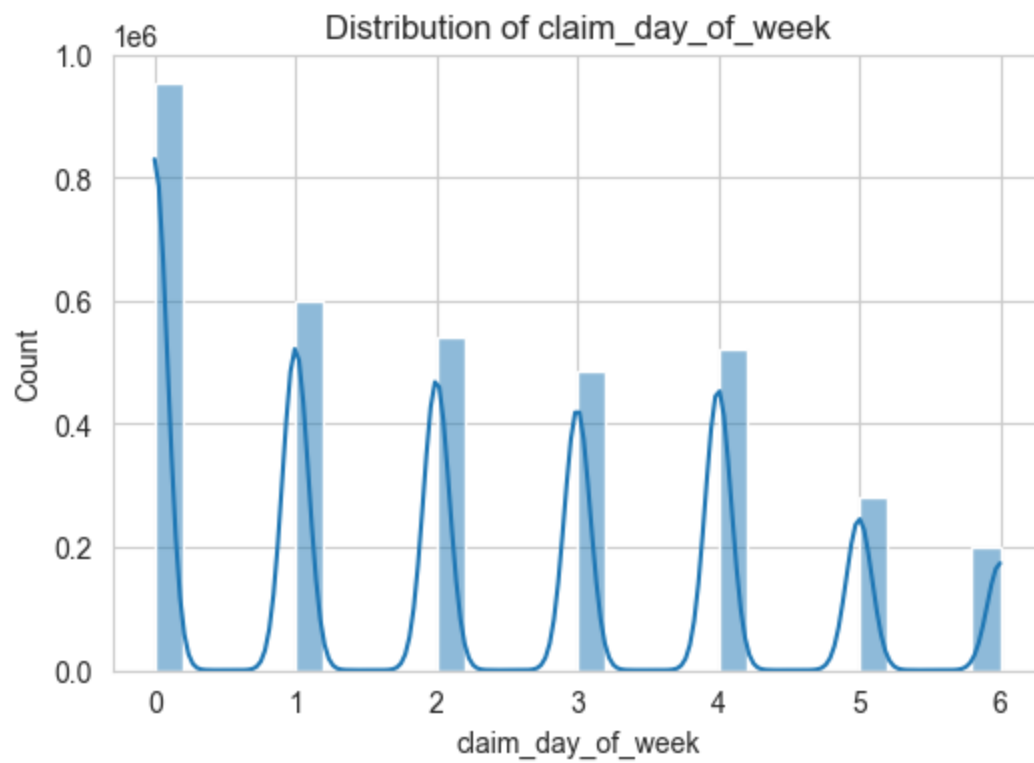
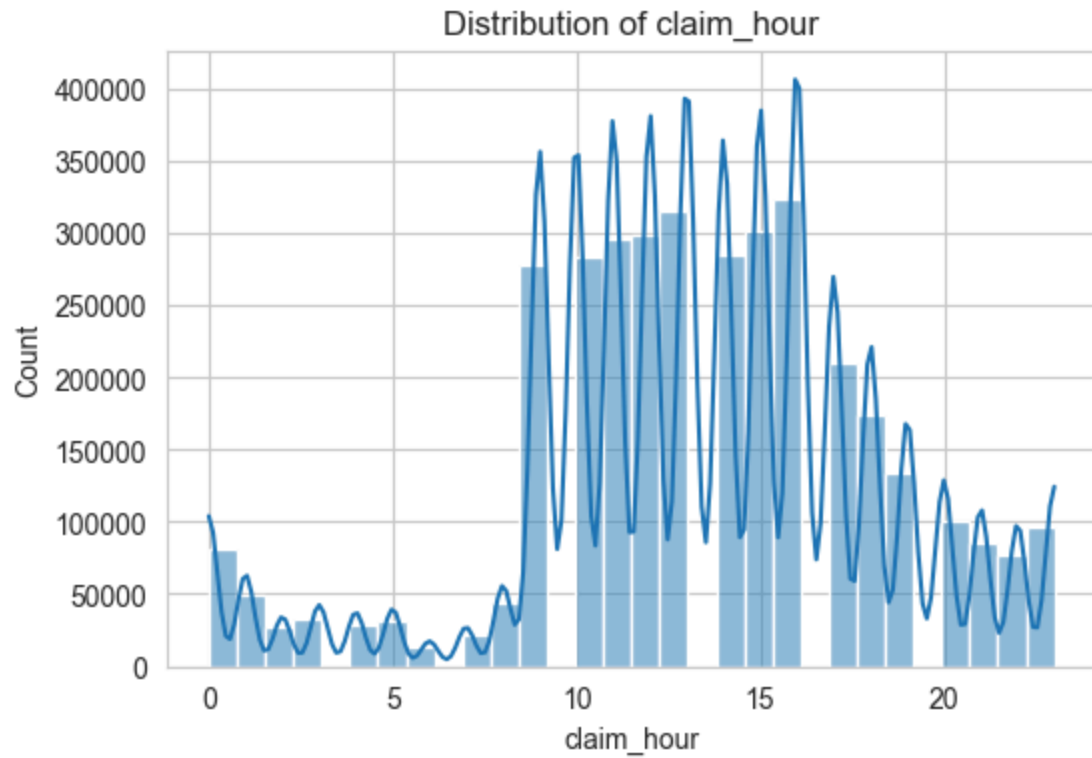
```
In [23]: # Numeric features
for col in num_features:
    plt.figure(figsize=(6,4))
    sns.histplot(X[col], kde=True, bins=30)
    plt.title(f"Distribution of {col}")
    plt.show()

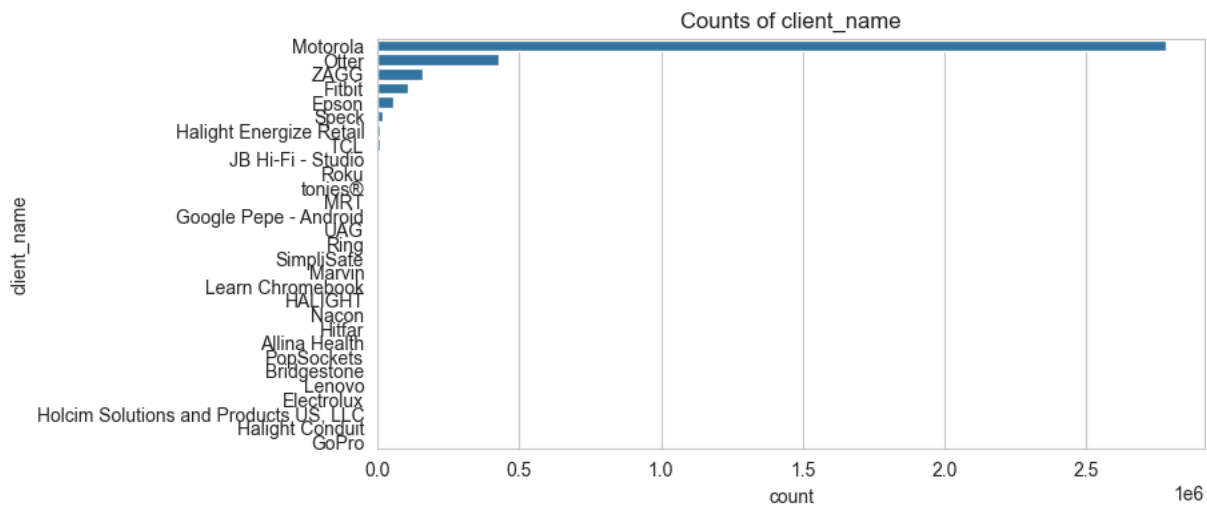
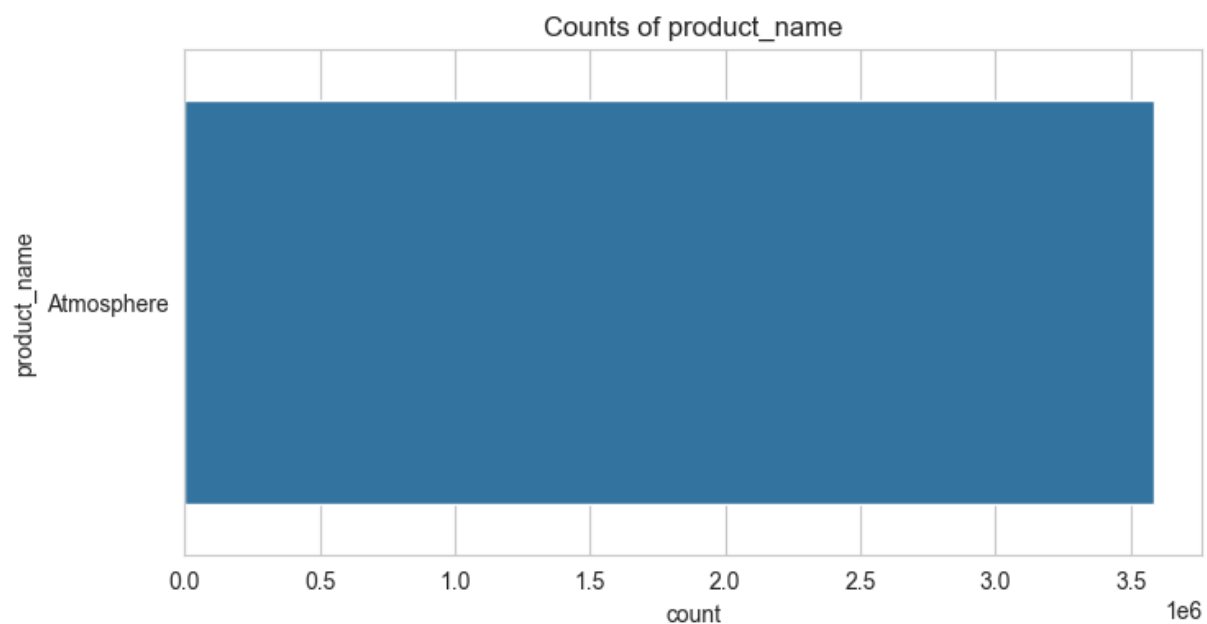
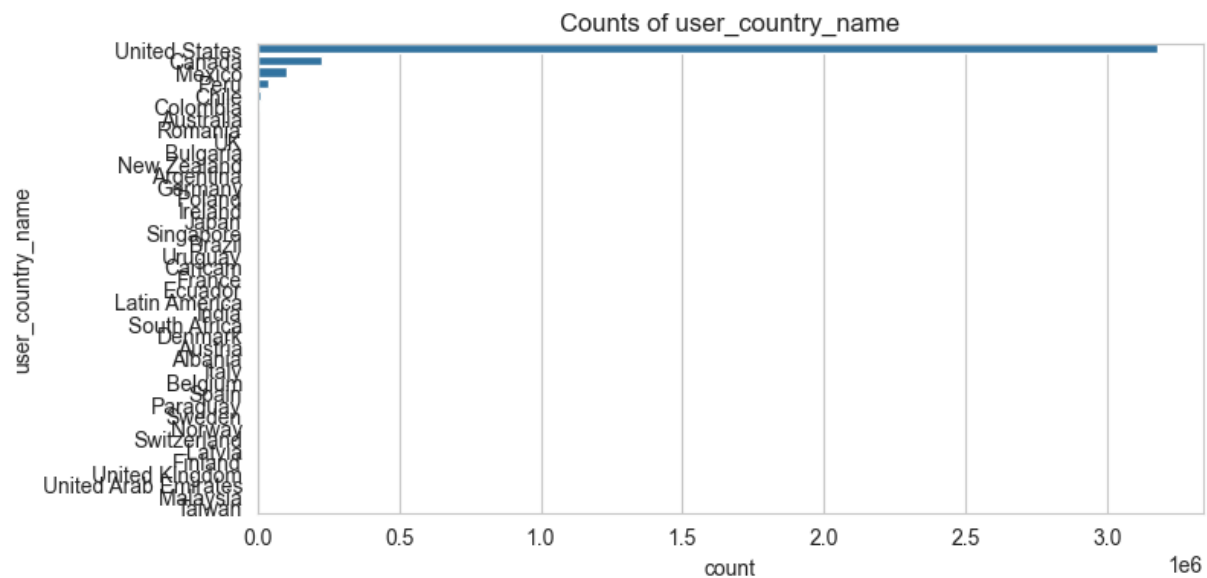
# Categorical features
for col in cat_features:
    plt.figure(figsize=(8,4))
    sns.countplot(y=X[col], order=X[col].value_counts().index)
    plt.title(f"Counts of {col}")
    plt.show()

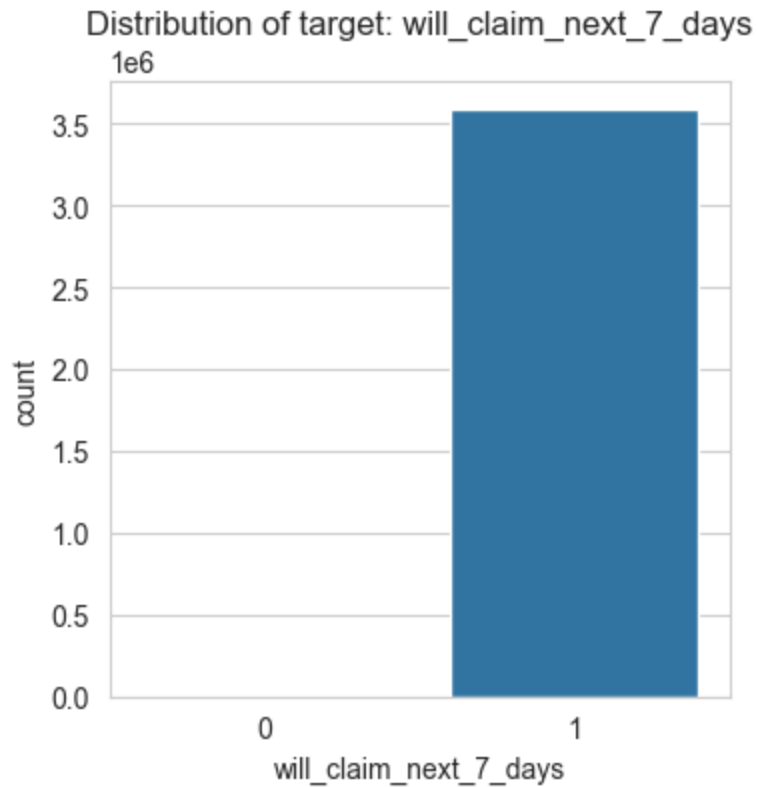
# Target distribution
plt.figure(figsize=(4,4))
sns.countplot(x=y)
plt.title("Distribution of target: will_claim_next_7_days")
plt.show()
```







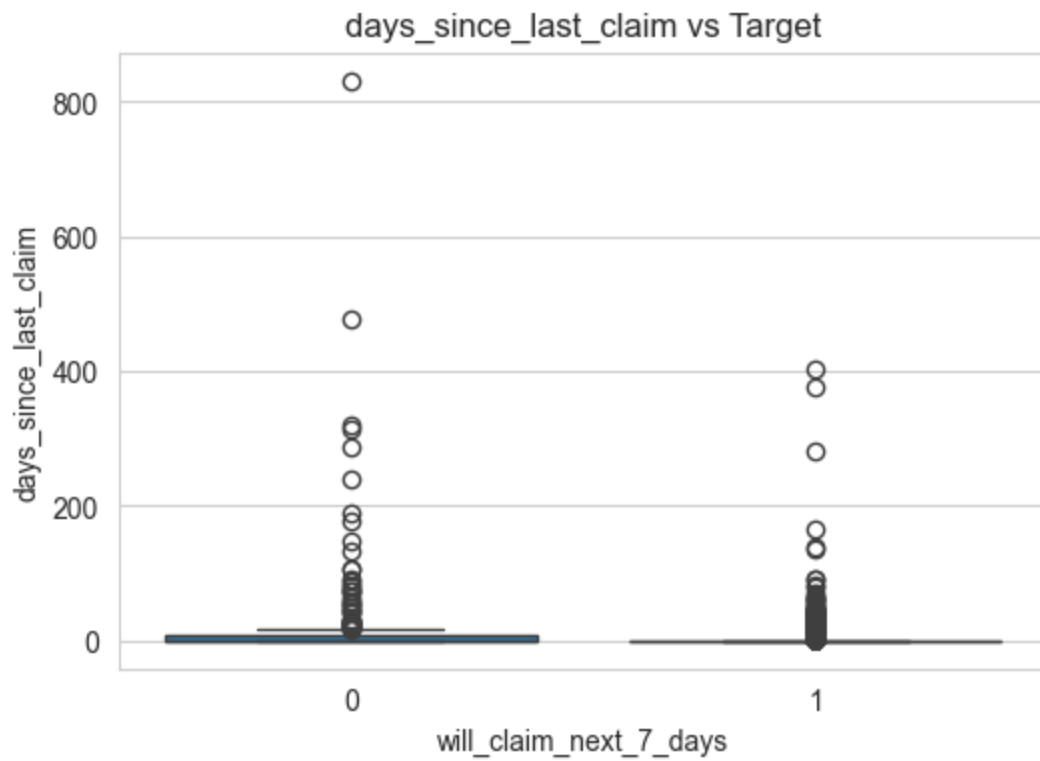
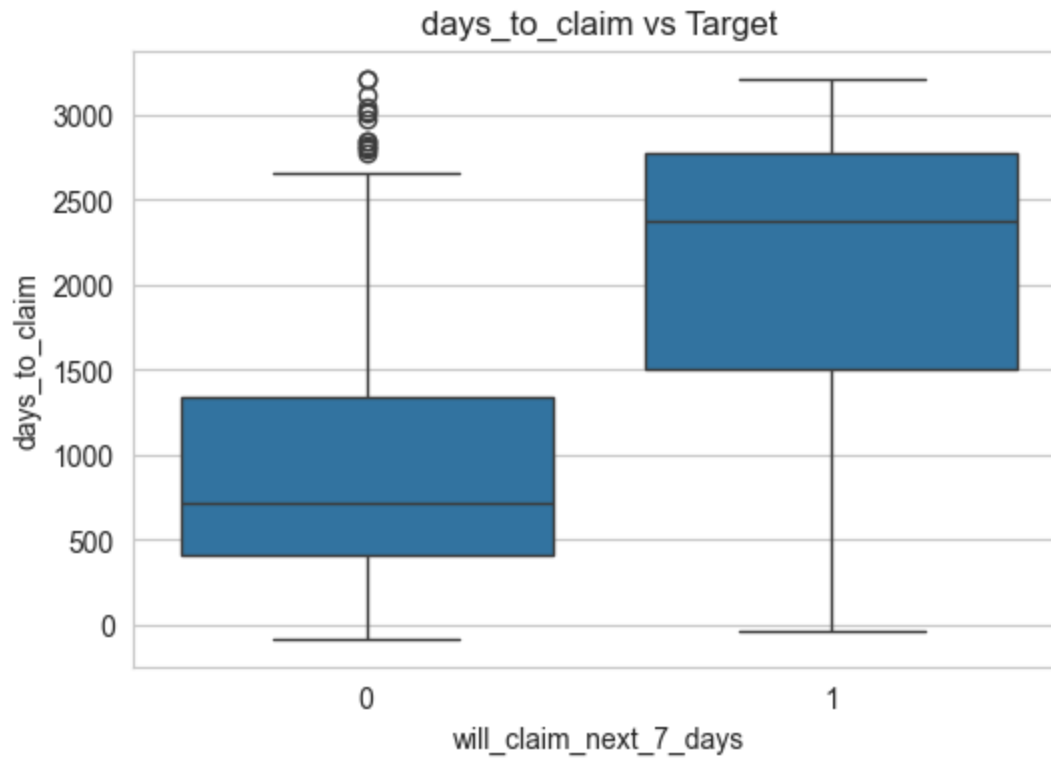


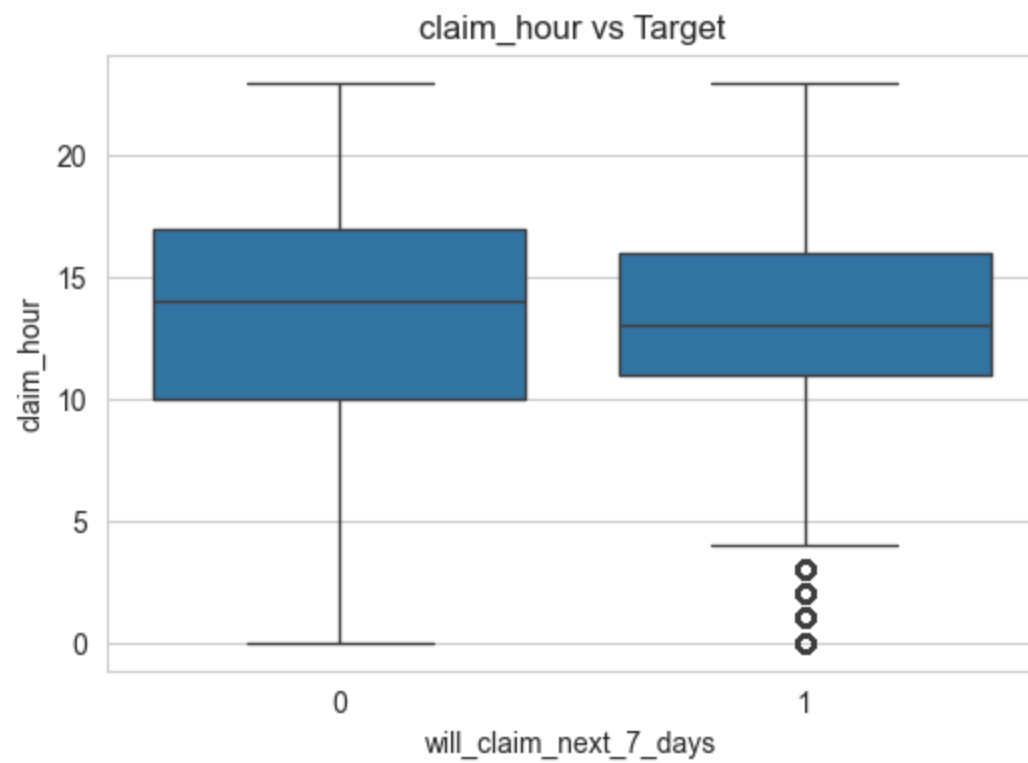
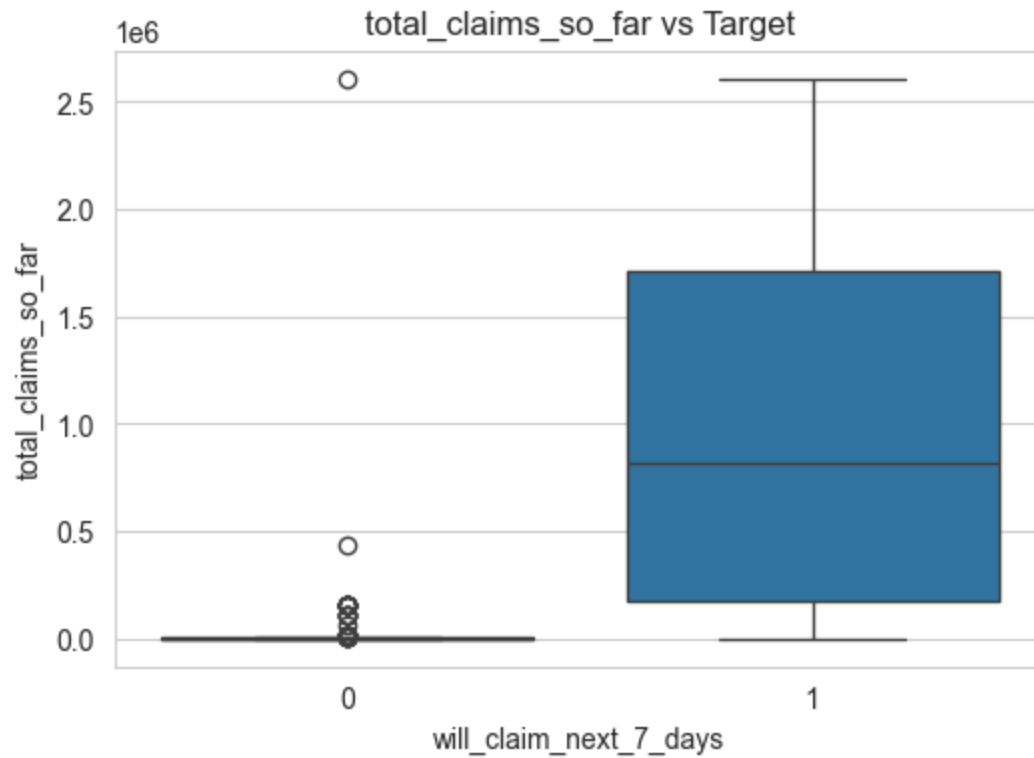


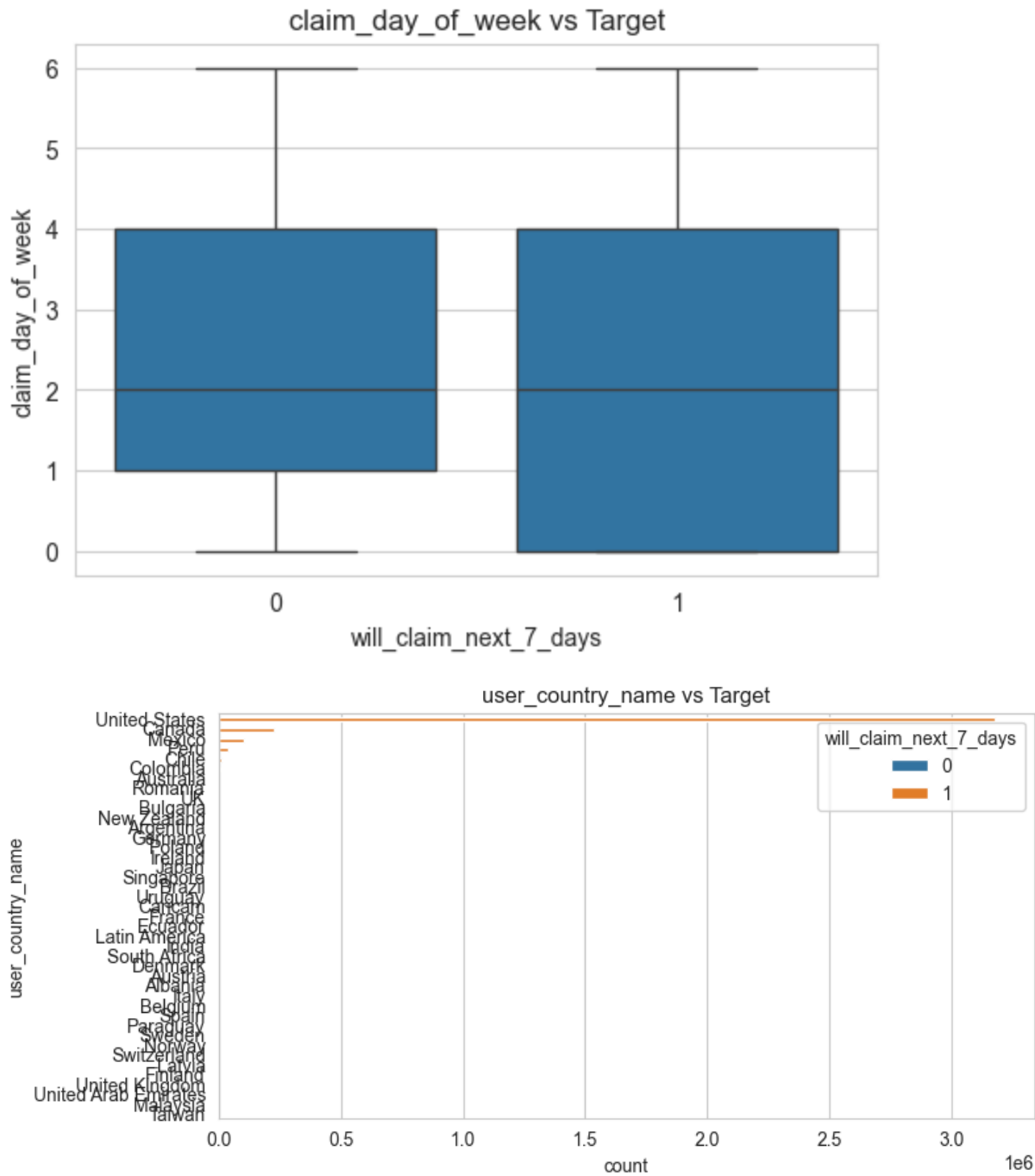
Bivariate Analysis (Feature vs Target)

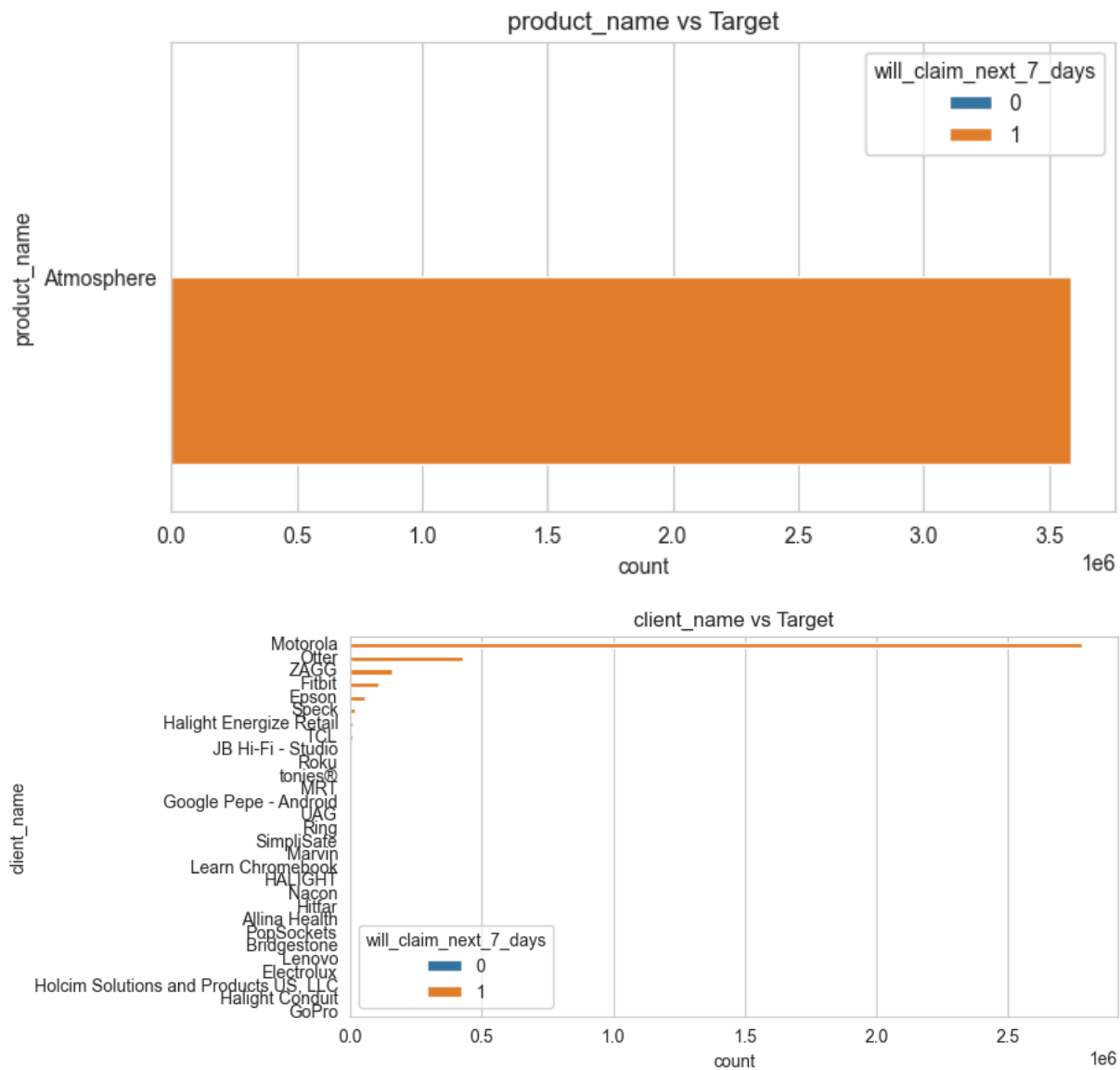
```
In [24]: # Numeric vs Target
for col in num_features:
    plt.figure(figsize=(6,4))
    sns.boxplot(x=y, y=X[col])
    plt.title(f"{col} vs Target")
    plt.show()

# Categorical vs Target
for col in cat_features:
    plt.figure(figsize=(8,4))
    sns.countplot(y=X[col], hue=y, order=X[col].value_counts().index)
    plt.title(f"{col} vs Target")
    plt.show()
```



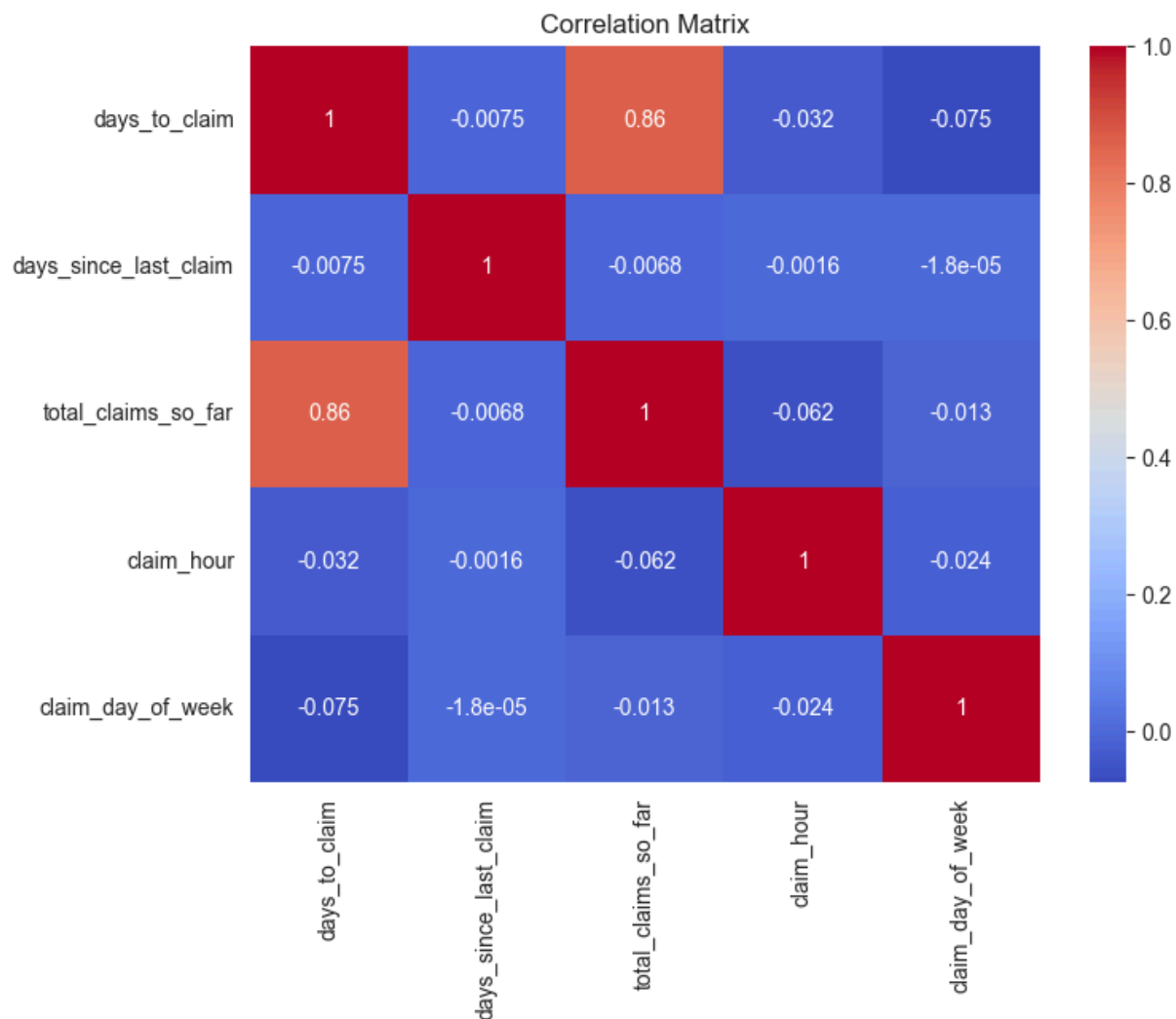






In [25]: *#Correlation heatmap*

```
plt.figure(figsize=(8,6))
sns.heatmap(X[num_features].corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



Prepare features & target

```
In [26]: target = "will_claim_next_7_days"

num_features = ["days_to_claim", "days_since_last_claim", "total_claims_so_far", "c
cat_features = ["user_country_name", "product_name", "client_name"]

X = df[num_features + cat_features]
y = df[target]
```

Split data

```
In [27]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Fix dtypes

```
In [28]: # Numeric columns → float (pd.NA → np.nan)
for col in num_features:
    X_train[col] = pd.to_numeric(X_train[col], errors="coerce")
    X_test[col] = pd.to_numeric(X_test[col], errors="coerce")
```

```
# Categorical columns → string
for col in cat_features:
    X_train[col] = X_train[col].astype("string")
    X_test[col] = X_test[col].astype("string")
```

```
In [31]: # -----
# 1. Ensure numeric columns are float
# -----
for col in num_features:
    X_train[col] = pd.to_numeric(X_train[col], errors="coerce")
    X_test[col] = pd.to_numeric(X_test[col], errors="coerce")

# -----
# 2. Ensure categorical columns are string
# -----
for col in cat_features:
    X_train[col] = X_train[col].astype(str).fillna("Unknown")
    X_test[col] = X_test[col].astype(str).fillna("Unknown")
```

Preprocessing & pipeline

```
In [32]: numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, num_features),
    ("cat", categorical_transformer, cat_features)
])

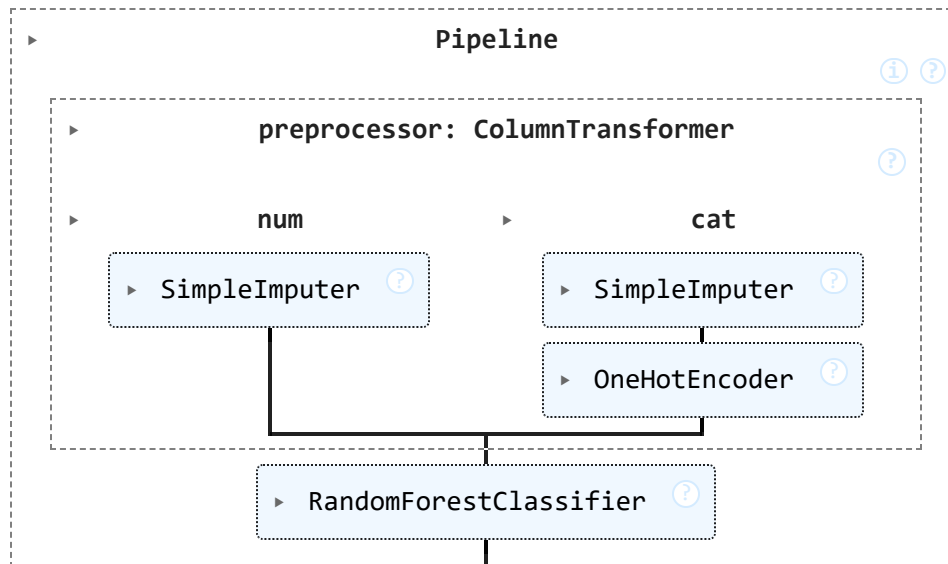
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", model)
])
```

Train model

```
In [33]: pipeline.fit(X_train, y_train)
```

Out[33]:



Evaluate model

```

In [38]: preds = pipeline.predict(X_test)
probs = pipeline.predict_proba(X_test)[: , 1]

print(classification_report(y_test, preds))
print("ROC AUC:", roc_auc_score(y_test, probs))

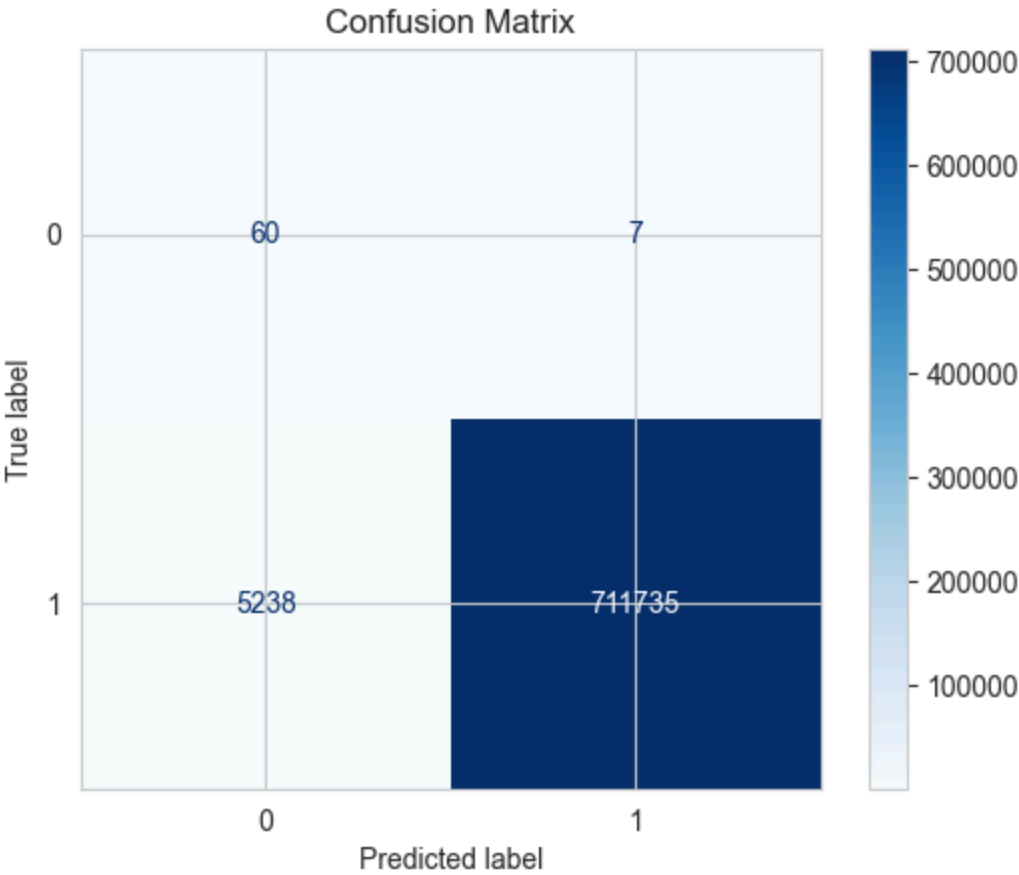
# Compute confusion matrix
cm = confusion_matrix(y_test, preds)

# Display
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=pipeline.classes_)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()

```

	precision	recall	f1-score	support
0	0.01	0.90	0.02	67
1	1.00	0.99	1.00	716973
accuracy			0.99	717040
macro avg	0.51	0.94	0.51	717040
weighted avg	1.00	0.99	1.00	717040

ROC AUC: 0.9967673692660339



```
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
```